

Objetivos da aula:

- Compreender os conceitos de coesão e acoplamento no desenvolvimento de software.
- Identificar problemas de organização de código relacionados à baixa coesão e alto acoplamento.
- Aplicar princípios básicos de organização de classes e módulos em TypeScript.
- Entender como esses princípios influenciam a arquitetura baseada em componentes utilizada no React.
- Reconhecer a importância da separação de responsabilidades na construção de aplicações modernas.

Índice:

- i. Coesão
- ii. Acoplamento
- iii. Relação entre Coesão e Acoplamento
- iv. Componentização e Arquitetura em React

i. Coesão

1.1. Introdução

No desenvolvimento de software, a forma como organizamos o código influencia diretamente a **qualidade, manutenção e evolução de um sistema**.

À medida que os sistemas crescem, torna-se cada vez mais importante dividir o software em partes menores e bem definidas. Essas partes podem ser representadas por:

- classes
- módulos
- componentes
- serviços

Para organizar corretamente essas estruturas, a engenharia de software utiliza alguns princípios fundamentais de design. Um dos mais importantes é o conceito de **coesão**.

A coesão está relacionada à **organização interna de um módulo ou classe**, indicando o quanto seus elementos estão relacionados entre si para cumprir um objetivo específico.

1.2. Definição de Coesão

A **coesão** pode ser definida como o **grau de relacionamento entre as responsabilidades de um módulo, classe ou componente**.

Em outras palavras, a coesão mede **o quanto os elementos de uma estrutura de software trabalham juntos para cumprir uma mesma finalidade**.

Quando uma classe ou módulo possui alta coesão, significa que:

- suas funções estão relacionadas

- sua responsabilidade é clara
- seu objetivo dentro do sistema é bem definido

Por outro lado, quando uma estrutura possui **baixa coesão**, suas funcionalidades tendem a ser pouco relacionadas entre si, o que pode dificultar a compreensão e manutenção do código.

1.3. Responsabilidade de Classes

Um dos princípios mais importantes relacionados à coesão é o conceito de **responsabilidade**.

Cada classe ou módulo deve possuir **uma responsabilidade principal dentro do sistema**.

Isso significa que a classe deve representar **um único conceito ou função do domínio da aplicação**.

Por exemplo:

Classe	Responsabilidade
UsuarioService	gerenciamento de usuários
EmailService	envio de e-mails
RelatorioService	geração de relatórios

Quando uma classe possui múltiplas responsabilidades diferentes, a coesão tende a diminuir.

1.4. Exemplo de Baixa Coesão

Considere o exemplo a seguir em TypeScript:

```
class Sistema {
  salvarUsuario(usuario: string) {
    console.log("Usuário salvo:", usuario);
  }

  gerarRelatorio() {
    console.log("Gerando relatório...");
  }

  enviarEmail(email: string) {
    console.log("Enviando email para:", email);
  }

  calcularPagamento(valor: number) {
    return valor * 1.1;
  }
}
```

Essa classe reúne diversas responsabilidades diferentes:

- gerenciamento de usuários
- geração de relatórios
- envio de e-mails

- cálculos financeiros

Essas funcionalidades pertencem a **partes distintas de um sistema**, o que caracteriza um caso de **baixa coesão**.

Quando classes possuem muitas responsabilidades, alguns problemas podem surgir:

- dificuldade de manutenção
- dificuldade de reutilização
- maior complexidade do código
- aumento da chance de erros

1.5. Melhorando a Coesão

Uma forma de melhorar a coesão é **separar as responsabilidades em diferentes classes**, cada uma responsável por uma parte específica do sistema.

Exemplo:

```
class UsuarioService {
    salvarUsuario(usuario: string) {
        console.log("Usuário salvo:", usuario);
    }
}

class RelatorioService {
    gerarRelatorio() {
        console.log("Gerando relatório...");
    }
}

class EmailService {
    enviarEmail(email: string) {
        console.log("Enviando email para:", email);
    }
}

class PagamentoService {
    calcularPagamento(valor: number) {
        return valor * 1.1;
    }
}
```

Nesse caso, cada classe possui **uma responsabilidade bem definida**, o que aumenta a coesão do sistema.

Essa organização também torna o código:

- mais legível
- mais fácil de manter
- mais fácil de reutilizar

1.6. Coesão em Aplicações Modernas

O conceito de coesão não se aplica apenas a classes em linguagens orientadas a objetos. Ele também é utilizado em **arquiteturas modernas de desenvolvimento**, como aplicações front-end desenvolvidas com **React e TypeScript**.

Nessas aplicações, a interface é dividida em **componentes**, e cada componente deve possuir uma responsabilidade clara dentro da aplicação.

Por exemplo, em um sistema de gerenciamento de usuários podemos ter componentes como:

- formulário de cadastro de usuário
- lista de usuários
- visualização de perfil

Cada um desses componentes representa **uma parte específica da interface**, o que contribui para manter a organização do sistema.

Nos próximos capítulos veremos como esse princípio se relaciona com outros conceitos importantes de arquitetura de software.

1.7 Boas Práticas para Alta Coesão

Algumas práticas ajudam a manter um sistema com **alta coesão**:

- criar classes com responsabilidades bem definidas
- evitar classes ou módulos muito grandes
- dividir funcionalidades complexas em partes menores
- organizar o sistema em módulos ou componentes
- separar responsabilidades de interface e lógica de negócio

Essas práticas são amplamente utilizadas no desenvolvimento de aplicações modernas e contribuem para a construção de sistemas mais organizados e sustentáveis ao longo do tempo.

ii. Acoplamento

2.1. Introdução

No capítulo anterior foi apresentado o conceito de coesão, que está relacionado à organização interna de classes, módulos ou componentes.

Além de organizar bem as responsabilidades dentro de uma estrutura, também é importante observar **como diferentes partes do sistema se relacionam entre si**.

Esse relacionamento entre partes do software é descrito pelo conceito de **acoplamento**.

Enquanto a coesão analisa **o interior de um módulo**, o acoplamento analisa **a relação entre diferentes módulos ou classes**.

2.2. Definição de Acoplamento

O acoplamento refere-se ao **grau de dependência entre módulos, classes ou componentes de um sistema**.

Em termos práticos, o acoplamento mede **o quanto uma parte do sistema depende diretamente de outra para funcionar**.

Quando um sistema apresenta **alto acoplamento**, alterações em um módulo podem exigir mudanças em vários outros módulos.

Por outro lado, sistemas com **baixo acoplamento** tendem a possuir partes mais independentes, o que facilita a manutenção e evolução do software.

2.3. Consequências do Alto Acoplamento

Quando o acoplamento entre partes do sistema é elevado, alguns problemas podem surgir:

- alterações em uma classe afetam diversas outras partes do sistema
- maior dificuldade para testar o código de forma isolada
- menor reutilização de módulos
- aumento da complexidade do sistema

Por esse motivo, durante o desenvolvimento de software procura-se construir sistemas com **baixo acoplamento entre seus módulos**.

2.4. Exemplo de Alto Acoplamento em TypeScript

Considere o exemplo a seguir:

```
class EmailService {
    enviarEmail(email: string) {
        console.log("Email enviado para:", email);
    }
}

class UsuarioService {
    private emailService = new EmailService();

    cadastrar(email: string) {
        console.log("Usuário cadastrado");
        this.emailService.enviarEmail(email);
    }
}
```

Nesse exemplo, a classe `UsuarioService` cria diretamente uma instância da classe `EmailService`.

Isso faz com que `UsuarioService` dependa diretamente da implementação de `EmailService`.

Caso seja necessário alterar a forma de envio de e-mails, será necessário modificar também a classe `UsuarioService`.

Esse tipo de dependência direta caracteriza um caso de **alto acoplamento**.

2.5. Reduzindo o Acoplamento

Uma forma de reduzir o acoplamento é evitar que uma classe seja responsável por **criar diretamente suas dependências**.

Em vez disso, as dependências podem ser **recebidas por meio de parâmetros ou construtores**.

Exemplo:

```
class EmailService {
  enviarEmail(email: string) {
    console.log("Email enviado para:", email);
  }
}

class UsuarioService {
  constructor(private emailService: EmailService) {}

  cadastrar(email: string) {
    console.log("Usuário cadastrado");
    this.emailService.enviarEmail(email);
  }
}

const emailService = new EmailService();
const usuarioService = new UsuarioService(emailService);

usuarioService.cadastrar("usuario@email.com");
```

Nesse caso, a classe `UsuarioService` **não é responsável por criar o `EmailService`**, apenas utiliza um objeto que foi fornecido externamente.

Essa abordagem reduz o acoplamento entre as classes.

2.6. Acoplamento em Aplicações Front-End

Em aplicações modernas, especialmente em aplicações desenvolvidas com React e TypeScript, também é importante evitar dependências excessivas entre partes do sistema.

Um exemplo comum ocorre quando componentes acessam diretamente serviços ou dados externos.

Considere o exemplo a seguir:

Exemplo com alto acoplamento

```
function UserProfile() {
  const user = UserService.getUser();

  return <div>{user.name}</div>;
}
```

Nesse caso, o componente depende diretamente da implementação de `UserService`.

Se o mecanismo de obtenção de dados for alterado, esse componente também precisará ser modificado.

Uma abordagem mais flexível consiste em fornecer os dados necessários ao componente por meio de **propriedades (props)**.

```
type Props = {  
  name: string  
}  
  
function UserProfile({ name }: Props) {  
  return <div>{name}</div>  
}
```

Nesse caso, o componente passa a depender apenas dos dados que recebe, tornando-se mais reutilizável e menos dependente de outras partes do sistema.

2.7. Boas Práticas para Reduzir o Acoplamento

Algumas práticas ajudam a manter sistemas com **baixo acoplamento**:

- evitar dependências diretas entre classes sempre que possível
- separar responsabilidades em módulos diferentes
- utilizar parâmetros ou construtores para fornecer dependências
- dividir sistemas em partes menores e bem definidas
- organizar o código em camadas ou módulos

Essas práticas contribuem para a construção de sistemas mais flexíveis e mais fáceis de evoluir.

iii. Relação entre Coesão e Acoplamento

3.1. Introdução

Nos capítulos anteriores foram apresentados dois conceitos fundamentais no design de software:

- coesão
- acoplamento

Esses dois conceitos estão diretamente relacionados à forma como o código de um sistema é organizado.

Enquanto a **coesão** analisa a organização **interna de uma classe ou módulo**, o **acoplamento** analisa a **dependência entre diferentes partes do sistema**.

Para construir sistemas bem estruturados, é importante considerar esses dois aspectos em conjunto.

3.2. Alta Coesão e Baixo Acoplamento

Em projetos de software bem estruturados, busca-se alcançar a seguinte combinação:

Alta coesão + Baixo acoplamento

Essa combinação resulta em sistemas que apresentam diversas vantagens:

- melhor organização do código
- maior facilidade de manutenção
- maior possibilidade de reutilização de módulos
- maior facilidade de testes

Quando cada módulo possui uma responsabilidade clara e poucas dependências externas, o sistema tende a se tornar mais simples de compreender e evoluir.

3.3. Comparação entre os Conceitos

Embora os conceitos de coesão e acoplamento estejam relacionados, eles analisam aspectos diferentes da estrutura de um sistema.

Conceito	O que analisa
Coesão	Organização interna de uma classe ou módulo
Acoplamento	Dependência entre diferentes classes ou módulos

De forma simplificada:

- **coesão** trata da responsabilidade de uma estrutura
- **acoplamento** trata da dependência entre estruturas

Um bom design de software procura garantir que:

- cada módulo tenha uma responsabilidade bem definida
- os módulos dependam o mínimo possível uns dos outros

3.4. Exemplo de Estrutura com Problemas de Design

Considere o exemplo a seguir:

```
class Sistema {
    salvarUsuario(nome: string) {
        console.log("Usuário salvo:", nome);
    }

    enviarEmail(email: string) {
        console.log("Email enviado para:", email);
    }

    gerarRelatorio() {
        console.log("Relatório gerado");
    }
}
```

Nesse caso, a classe reúne várias responsabilidades diferentes, o que caracteriza **baixa coesão**.

Além disso, se outras partes do sistema dependerem diretamente dessa classe para executar essas funções, o sistema também poderá apresentar **alto acoplamento**.

Esse tipo de estrutura tende a dificultar a manutenção do sistema ao longo do tempo.

3.5. Melhorando a Organização do Sistema

Uma forma de melhorar a estrutura do sistema é separar as responsabilidades em classes distintas.

Exemplo:

```
class UsuarioService {
  salvarUsuario(nome: string) {
    console.log("Usuário salvo:", nome);
  }
}

class EmailService {
  enviarEmail(email: string) {
    console.log("Email enviado para:", email);
  }
}

class RelatorioService {
  gerarRelatorio() {
    console.log("Relatório gerado");
  }
}
```

Nesse caso:

- cada classe possui **uma responsabilidade específica**, aumentando a coesão
- as classes podem ser utilizadas de forma independente, reduzindo o acoplamento

Essa organização torna o sistema mais fácil de manter e evoluir.

3.6. Aplicação em Arquiteturas Modernas

Os princípios de **alta coesão e baixo acoplamento** são amplamente utilizados em arquiteturas modernas de software.

Eles aparecem em diversas abordagens de desenvolvimento, como:

- arquitetura em camadas
- arquitetura baseada em componentes
- desenvolvimento de aplicações web modernas

Essas abordagens buscam dividir o sistema em partes menores, onde cada parte possui uma responsabilidade específica dentro da aplicação.

No próximo capítulo veremos como esses princípios são aplicados na prática em aplicações desenvolvidas com React e TypeScript, que utilizam uma arquitetura baseada em componentes.

iv. Componentização e Arquitetura em React

4.1. Introdução

Nos capítulos anteriores foram apresentados dois princípios importantes de organização de software:

- **coesão**, relacionada à responsabilidade interna de uma classe ou módulo
- **acoplamento**, relacionado ao grau de dependência entre partes do sistema

Esses princípios não se aplicam apenas ao desenvolvimento orientado a objetos. Eles também influenciam diretamente a forma como aplicações modernas são estruturadas.

Um exemplo bastante conhecido é o **React**, que utiliza uma arquitetura baseada em **componentes**. Essa arquitetura busca dividir a interface da aplicação em partes menores e independentes, facilitando a organização e manutenção do código.

4.2. O Conceito de Componentes

Em aplicações React, a interface do sistema é construída a partir de **componentes**.

Um componente pode ser entendido como uma unidade que reúne:

- estrutura visual da interface
- comportamento
- dados necessários para a exibição

Cada componente representa uma parte específica da interface da aplicação.

Exemplos comuns de componentes incluem:

- botão
- formulário
- lista de usuários
- cartão de produto
- barra de navegação

Ao dividir a interface em componentes menores, o sistema torna-se mais organizado e mais fácil de evoluir.

4.3. Componentes em React com TypeScript

Em aplicações React que utilizam TypeScript, é comum definir explicitamente o tipo das propriedades que um componente recebe.

Observe o exemplo a seguir:

```
type Props = {  
  nome: string  
}  
  
function Saudacao({ nome }: Props) {  
  return <h1>Olá {nome}</h1>  
}
```

Nesse exemplo temos um componente chamado Saudacao, que recebe uma propriedade chamada nome.

A tipagem explícita das propriedades ajuda a tornar o código mais seguro e facilita a identificação de erros durante o desenvolvimento.

4.4. Responsabilidade de Componentes

Assim como ocorre com classes e módulos, os componentes também devem possuir **responsabilidades bem definidas**.

Quando um componente assume muitas responsabilidades diferentes, o código tende a se tornar mais difícil de compreender e manter.

Considere o exemplo a seguir:

```
function PaginaUsuario() {  
  
  // chamada de API  
  // validação de formulário  
  // lógica de autenticação  
  // renderização da interface  
  
  return <div>Página do usuário</div>  
}
```

Nesse caso, o componente concentra diversas responsabilidades diferentes, como acesso a dados, validação e renderização.

Esse tipo de estrutura tende a dificultar a manutenção do sistema.

4.5. Divisão da Interface em Componentes

Uma abordagem mais adequada consiste em dividir a interface em componentes menores, cada um responsável por uma parte específica da aplicação.

Exemplo:

```
components  
├─ UserForm.tsx  
├─ UserList.tsx  
└─ UserProfile.tsx
```

Nesse caso:

- `UserForm` pode ser responsável pelo cadastro de usuários
- `UserList` pode exibir a lista de usuários
- `UserProfile` pode apresentar informações detalhadas de um usuário

Essa divisão ajuda a manter a aplicação organizada e facilita a reutilização de componentes.

4.6. Organização de Projetos React

Em aplicações React, também é comum organizar o projeto em diferentes pastas para separar responsabilidades.

Uma estrutura típica pode ser:

```
src
├ components
├ services
├ hooks
├ models
└ utils
```

Cada parte da estrutura possui um papel específico no sistema.

Por exemplo:

- `components`: componentes da interface
- `services`: acesso a dados ou APIs
- `models`: definição de tipos e estruturas de dados
- `hooks`: lógica reutilizável baseada em hooks do React
- `utils`: funções auxiliares

Essa organização contribui para manter o código mais estruturado e compreensível.

4.7. Separação entre Interface e Dados

Outra prática comum em aplicações React é separar a lógica de acesso a dados da interface.

Por exemplo, um serviço pode ser responsável por acessar uma API:

```
export class UserService {
  async getUsers() {
    const response = await fetch("/api/users");

    return response.json();
  }
}
```

Enquanto isso, um componente pode ser responsável apenas pela exibição das informações:

```
type User = {
  id: number
  name: string
}

function UserList({ users }: { users: User[] }) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  )
}
```

```
    }}}  
  </ul>  
)  
}
```

Essa separação ajuda a manter o sistema mais organizado e facilita a manutenção do código.

4.8. Aplicação dos Conceitos da Aula

A arquitetura baseada em componentes utilizada pelo React aplica diretamente os princípios estudados nesta aula.

Ao dividir a aplicação em componentes menores e bem definidos, torna-se possível:

- organizar melhor o sistema
- reduzir dependências desnecessárias entre partes do código
- facilitar a manutenção da aplicação
- promover a reutilização de componentes

Esses princípios são fundamentais para o desenvolvimento de aplicações modernas utilizando React e TypeScript.

Exercícios

Veja os vídeos se tiver dúvidas:

Coesão e acoplamento: <https://youtu.be/O5PEI-tVmtM>

Exercícios: <https://youtu.be/NdhkHgcx5bU>

Exercício 1 – Identificando Baixa Coesão

Considere o código TypeScript a seguir:

```
class Pedido {  
  adicionarProduto(nome: string, preco: number) {  
    console.log("Produto adicionado:", nome, preco);  
  }  
  
  calcularTotal(valor: number) {  
    return valor * 1.1;  
  }  
  
  enviarEmailConfirmacao(email: string) {  
    console.log("Email de confirmação enviado para:", email);  
  }  
  
  gerarRelatorioVendas() {  
    console.log("Relatório de vendas gerado");  
  }  
}
```

```
}
```

Questões

- a) Explique por que a classe Pedido apresenta **baixa coesão**.
- b) Identifique **quais responsabilidades diferentes** estão presentes nessa classe.
- c) Reescreva o código separando as responsabilidades em **classes mais específicas**, melhorando a coesão do sistema.

Exercício 2 – Identificando e Reduzindo Alto Acoplamento (TypeScript)

Este exercício dá continuidade ao Exercício 1: depois de separar responsabilidades (coesão), o próximo passo é observar **como as classes se relacionam** e evitar dependências diretas desnecessárias (**acoplamento**).

Enunciado

Considere o código TypeScript a seguir:

```
class EmailService {
  enviarConfirmacao(email: string) {
    console.log("Email de confirmação enviado para:", email);
  }
}

class PedidoService {
  // dependência criada internamente
  private emailService = new EmailService();

  finalizarPedido(emailCliente: string) {
    console.log("Pedido finalizado");
    this.emailService.enviarConfirmacao(emailCliente);
  }
}
```

Questões

- a) Explique por que o código apresenta **alto acoplamento** entre PedidoService e EmailService.
- b) Refatore o código para reduzir o acoplamento, fazendo com que PedidoService **não crie diretamente** o EmailService.
- c) Crie um pequeno trecho de teste (instanciação e chamada) que demonstre o funcionamento após a refatoração.

Exercício 3 – Baixo Acoplamento em React + TypeScript usando Props (ponte direta com React)

Este exercício mantém a progressão didática:

1. Exercício 1: baixa coesão → separar responsabilidades
2. Exercício 2: alto acoplamento entre classes → injeção de dependência no TypeScript

3. Exercício 3: aplicar o mesmo raciocínio no React: evitar que o componente dependa diretamente de um “service” e passar dependências via props.

Enunciado

Considere o componente React + TypeScript a seguir (acoplamento alto):

```
type User = {
  id: number;
  name: string;
  email: string;
};

class EmailService {
  enviarConfirmacao(email: string) {
    console.log("Email de confirmação enviado para:", email);
  }
}

function FinalizarPedidoButton({ user }: { user: User }) {
  const emailService = new EmailService();

  function finalizarPedido() {
    console.log("Pedido finalizado para:", user.name);
    emailService.enviarConfirmacao(user.email);
  }

  return <button onClick={finalizarPedido}>Finalizar pedido</button>;
}
```

Questões

- Explique por que o componente FinalizarPedidoButton está **fortemente acoplado** a EmailService.
- Refatore o componente para reduzir o acoplamento, removendo a criação do EmailService de dentro do componente.

Requisito: o componente deve receber por **props** uma função onConfirmacao(email: string): void.

- Crie um componente App que utilize FinalizarPedidoButton, implementando a função onConfirmacao fora do componente do botão.

Exercício 4 – Separação em Service + Hook + Componente (React + TypeScript)

Se no Exercício 3 você reduziu acoplamento com **props**, agora você organizará um fluxo típico de React:

- service (acesso a dados)
- hook (lógica e estado)
- component (apresentação)

Isso reforça: **alta coesão** (cada parte com um papel claro) e **baixo acoplamento** (interface desacoplada da fonte de dados).

Enunciado

Implemente uma listagem de usuários seguindo a estrutura:

```
src/  
  services/userService.ts  
  hooks/useUsers.ts  
  components/UserList.tsx  
  App.tsx
```

Considere os requisitos:

1. UserService deve possuir um método getUsers(): Promise<User[]> que retorna uma lista simulada de usuários (sem API real).
2. O hook useUsers deve:
 - buscar os usuários chamando o service
 - controlar users, loading e error
3. O componente UserList deve ser **apenas de apresentação**:
 - recebe users por props
 - renderiza a lista
4. O App deve:
 - usar o hook useUsers
 - exibir “Carregando...” quando loading for true
 - exibir mensagem de erro quando error existir
 - renderizar UserList quando os dados estiverem prontos

Tipos base: Use o tipo a seguir em todo o exercício (arquivo `models/User.ts`):

```
export type User = {  
  id: number  
  name: string  
  email: string  
}
```

Exercício 5 – Refatoração de UI: quando criar um componente? (React + TypeScript)**Enunciado**

Considere o componente UserList a seguir. Ele funciona, mas está com **baixa coesão** porque mistura a responsabilidade de:

- listar usuários
- definir a apresentação detalhada de cada usuário (card)

```
import { type User } from "../models/User";
```

```
type UserListProps = {
```

```
    users: User[];
  };

  export function UserList({ users }: UserListProps) {
    return (
      <div>
        {users.map((u) => (
          <div
            key={u.id}
            style={{
              border: "1px solid #ccc",
              padding: 12,
              marginBottom: 8,
              borderRadius: 6,
            }}
          >
            <h3>{u.name}</h3>
            <p>{u.email}</p>
            <button onClick={() => alert(`Usuário: ${u.name}`)}>
              Ver detalhes
            </button>
          </div>
        ))}
      </div>
    );
  }
}
```

Requisitos

a) Crie um novo componente UserCard em components/UserCard.tsx que receba por props:

- user: User
- onDetails: (user: User) => void

b) Refatore UserList para que ele:

- apenas percorra o array e renderize UserCard
- repasse a função onDetails para o UserCard

c) No App.tsx, implemente a função onDetails para exibir no console:

Detalhes do usuário: <nome> (<email>)